MPFT: Multipath Reliable File Transfer

Computer Networks project. A file is split into chunks, sent in parallel over several TCP connections ("paths"), and written on the receiver only after SHA-256 verification. The transfer survives a dead path, corrupted data and interruption (resume).

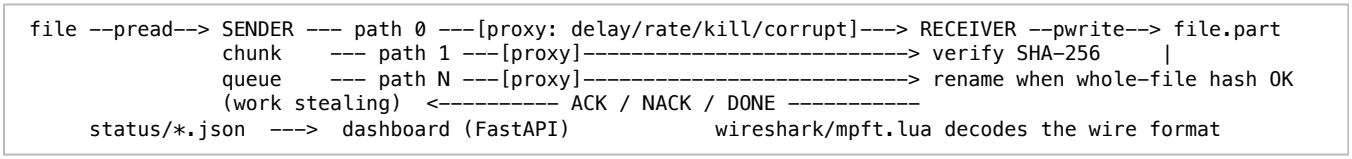
Problem statement

A single TCP connection ties a transfer to one route. If that route stalls or breaks the transfer stops, a damaged file is only discovered at the end, and an interrupted transfer starts again from zero. Most hosts have more than one usable route (WiFi plus hotspot, several connections). MPFT uses them together and recovers from failures at chunk level.

Objectives

1. Aggregate bandwidth by sending chunks over N parallel TCP paths with automatic load balancing.
2. Detect and repair corruption per chunk (SHA-256, NACK, retransmit).
3. Survive path failure by requeueing in-flight chunks on the remaining paths.
4. Resume interrupted transfers without resending verified data.
5. Measure all of this with an emulated network, graphs, a dashboard and Wireshark.

Architecture



Details: docs/ARCHITECTURE.md, docs/TOPOLOGY.md.

Protocol summary

20-byte big-endian header (`MP`, version, type, session id, chunk index, payload length, CRC-32 of the header), then payload. Types: HELLO, MANIFEST (JSON with chunk hashes), MANIFEST_ACK (chunks already held), CHUNK, ACK, NACK, DONE, ERROR. Paths pull chunk indexes from a shared queue (faster paths carry more). Full spec: docs/PROTOCOL.md.

Requirements

macOS (or any OS with Python), Python 3.11+. Core uses only the standard library. Extras (`requirements.txt`): pytest, matplotlib, fastapi, uvicorn, markdown, httpx.

Quick start

```
python3 -m venv .venv && source .venv/bin/activate && pip install -r requirements.txt
head -c 50000000 /dev/urandom > /tmp/big.bin

python -m mpft receive --listen 127.0.0.1:9000 --out downloads        # terminal 1
python -m mpft send /tmp/big.bin --to 127.0.0.1:9000 --paths 4        # terminal 2
shasum -a 256 /tmp/big.bin downloads/big.bin
```

Flags: `--paths N`, `--chunk-size 256k`, `--to host:port[,host:port...]`, `--listen`, `--out`, `--bind`, `--csv file`.

Emulated paths

```
python -m mpft emulate --config topology.example.json     # ports 9201-9203 forward to the receiver on 9000
python -m mpft send /tmp/big.bin --to 127.0.0.1:9201,127.0.0.1:9202,127.0.0.1:9203
```

Per path: `delay_ms`, `jitter_ms`, `rate_mbit`, `drop_after_s` (path dies), `corrupt_rate`.

Two Macs over Tailscale

Receiver: `python -m mpft receive --listen 0.0.0.0:9000 --out downloads`. Sender: `python -m mpft send file --to 100.x.y.z:9000 --paths 4`. Setup, firewall prompt, hotspot fallback and caveats about college WiFi and CGNAT: docs/REMOTE.md. (Not yet tested on real hardware.)

Web interface

One Mac, two instances and two browser tabs:

```
python -m mpft web --name alice --ui-port 8000 --peer-port 8001 --data-port 9000 --out ./alice-dl --host 127.0.
python -m mpft web --name bob   --ui-port 8010 --peer-port 8011 --data-port 9010 --out ./bob-dl   --host 127.0.
```

Two Macs: each person runs `python -m mpft web --name NAME --out ./downloads` (with Tailscale installed) and opens `http://127.0.0.1:8000`. The receiver copies the Receiver ID from the Receive page to the sender, who pastes it on the Send page. Both people then watch the same live Transfer page: chunk map, paths, graphs, incidents, hash checks and protocol messages, and past transfers can be replayed from History. No built-in encryption; Tailscale (WireGuard) encrypts between the Macs. Details, runbooks and troubleshooting: docs/WEB.md.

Dashboard

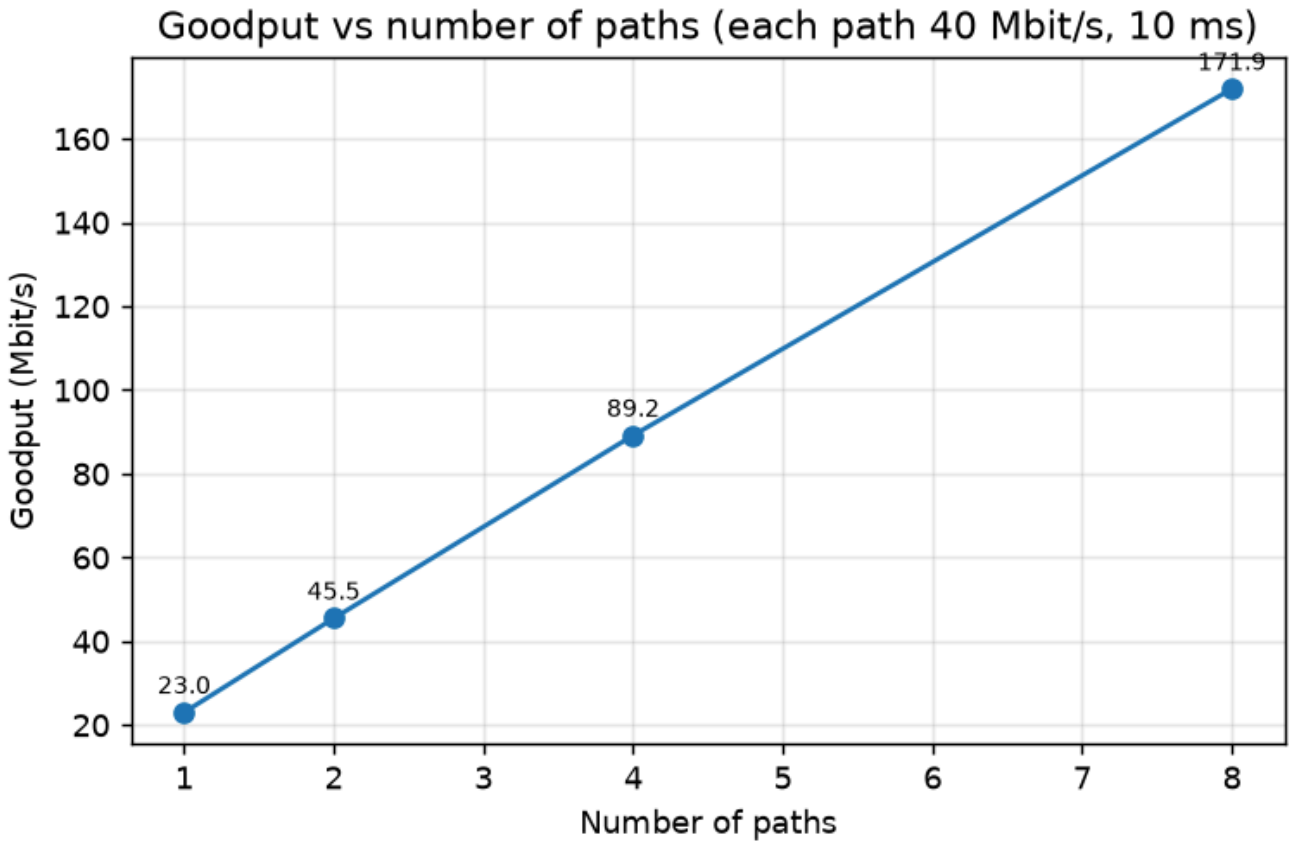
```
python -m mpft dashboard          # http://127.0.0.1:8080, reads status/*.json
```

Tests and experiments

```
pytest -q                                          # 20 tests, about 12 s
python experiments/run_experiments.py              # 50 MB, about 3 minutes; --size-mb, --only baseline,failure
python experiments/plot_results.py                 # PNGs into results/
```

Results (50 MB, emulated paths, one run each)

Experiment	Result
1 / 2 / 4 / 8 paths	23 / 45 / 89 / 172 Mbit/s
Paths 80/20/5 Mbit/s	bytes carried 74 % / 20 % / 6 %
One of 3 paths dies at 1.5 s	completes, SHA-256 verified, recovery 0.15 s
1 % and 5 % chunk corruption	2 and 12 retransmissions, all verified
Resume after kill at 40 %	80 of 200 chunks skipped, 5.7 s vs 9.2 s



Full tables and analysis: docs/TEST_REPORT.md. Raw data: `results/*.csv`.

Wireshark

`wireshark/mpft.lua` decodes MPFT (`mpft.type`, `mpft.session`, `mpft.chunk`, `mpft.len`, `mpft.crc`). Capture instructions and screenshot checklist: wireshark/CAPTURE.md.

Demo

5-minute script with exact commands: docs/DEMO_SCRIPT.md. Viva questions: docs/VIVA_PREP.md. Code tour: docs/WALKTHROUGH.md.

Limitations

- Paths are emulated on one machine; results are not real-network measurements.
- The emulator does not simulate true packet loss (TCP would hide it); corruption is injected above TCP.
- No encryption or authentication by default (use Tailscale/VPN).
- One chunk in flight per path (stop-and-wait), so a single path under-uses its link.
- Experiments ran once per setting, without confidence intervals.
- The two-Mac Tailscale run has not been tested on real hardware.

Team contributions

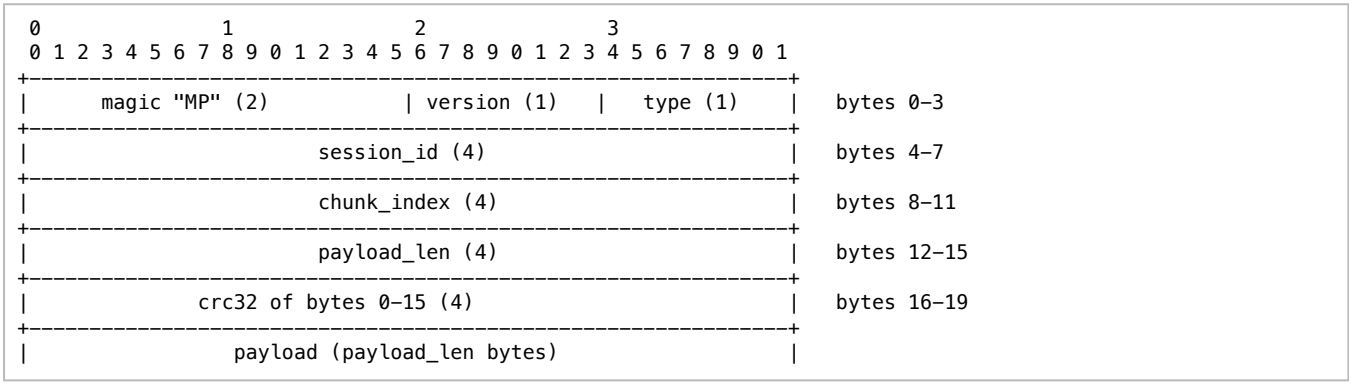
See docs/CONTRIBUTIONS.md.

MPFT Protocol

MPFT runs over ordinary TCP connections. Each TCP connection is a **path**. All integers are big-endian. Implementation: `mpft/protocol.py` (framing), `mpft/sender.py`, `mpft/receiver.py`.

1. Frame format

Every message is a 20-byte header followed by `payload_len` bytes of payload.



Field	Size	Meaning
magic	2	ASCII `MP`; anything else is rejected
version	1	1
type	1	message type, table below
session_id	4	random 32-bit id chosen by the sender, identifies one transfer on all its paths
chunk_index	4	chunk number for CHUNK/ACK/NACK, 0 otherwise
payload_len	4	payload bytes after the header (limit 64 MiB)
crc32	4	CRC-32 of the first 16 header bytes (detects a damaged header)

2. Message types

Type	Name	Direction	Payload
1	HELLO	sender to receiver	none; first message on every new connection, carries the session id
2	MANIFEST	sender to receiver	JSON `{name, size, chunk_size, total_chunks, file_sha256, chunk_sha256[]}`
3	MANIFEST_ACK	receiver to sender	JSON list of chunk indexes the receiver already holds (resume)
4	CHUNK	sender to receiver	raw bytes of chunk `chunk_index`
5	ACK	receiver to sender	none; `chunk_index` = chunk stored and verified
6	NACK	receiver to sender	none; `chunk_index` = chunk failed verification, resend

Type	Name	Direction	Payload
7	DONE	both	sender to receiver: "all chunks acknowledged, please verify". Receiver to sender: "file verified and renamed"
8	ERROR	receiver to sender	text reason (no manifest, hash mismatch, incomplete)

Chunk `i` covers file bytes `[i*chunk_size, min((i+1)*chunk_size, size))`. The last chunk may be shorter.

3. Sequence of a transfer (2 paths)

```
Sender (path 0)        Sender (path 1)                      Receiver
     | --- HELLO(sid) ------------------------------------------>|
     | --- MANIFEST(json) --------------------------------------->|  open file.part, scan it (resume)
     |<-------------------------------- MANIFEST_ACK([held idx]) ---|
     |                          | --- HELLO(sid) ---------------->|
     | --- CHUNK 0 ---------------------------------------------->|  check len + SHA-256, pwrite
     |                          | --- CHUNK 1 ------------------->|
     |<------------------------------------------------ ACK 0 ---|
     |                          |<------------------------ ACK 1 ---|
     | --- CHUNK 2 (corrupted) ---------------------------------->|  SHA-256 mismatch
     |<------------------------------------------------ NACK 2 ---|
     | --- CHUNK 2 (retransmit) --------------------------------->|
     |<------------------------------------------------ ACK 2 ---|
     |    ... all chunks ACKed ...                                |  last chunk: whole-file SHA-256
     | --- DONE ------------------------------------------------->|  verify, rename file.part -> file
     |<------------------------------------------------ DONE -|
```

Whichever path connects first sends the MANIFEST; the other paths wait until the MANIFEST_ACK is known (`Sender.ensure_manifest`). Each path then keeps exactly one CHUNK in flight (stop-and-wait per path).

4. State machines

Sender, per path worker (`Sender.worker`):

```
CONNECT --ok--> HANDSHAKE --> IDLE --pop chunk--> SENDING --ACK--> IDLE
   |  fail x4      |  fail                          |  NACK: requeue chunk, stay connected
   v               v                                |  timeout / reset / bad frame:
 DOWN <---- backoff + reconnect <--------------------+  requeue chunk, close, reconnect
 (after 6 consecutive failures the path stays DOWN)
any state --all chunks ACKed or fatal error--> STOP
```

Sender, whole transfer (`Sender.run`): `HASHING -> TRANSFERRING -> all ACKed -> DONE handshake -> SUCCESS`, or `-> FAILED` if a chunk exceeds 5 retries, all paths are down, or the receiver answers ERROR.

Receiver, per connection (`ReceiverHandle._serve`):

```
WAIT_HELLO -> READY --MANIFEST--> (open/scan session, reply MANIFEST_ACK) --> READY
READY --CHUNK--> verify -> ACK | NACK --> READY
READY --DONE---> finalize -> DONE | ERROR --> READY
any bad frame (magic, version, CRC, length) or socket error -> close connection
```

Receiver, per session (`Session`): `PENDING --last missing chunk stored--> whole-file SHA-256 --> VERIFIED (file.part renamed)` or `MISMATCH` (file.part kept, never renamed).

5. Error rules

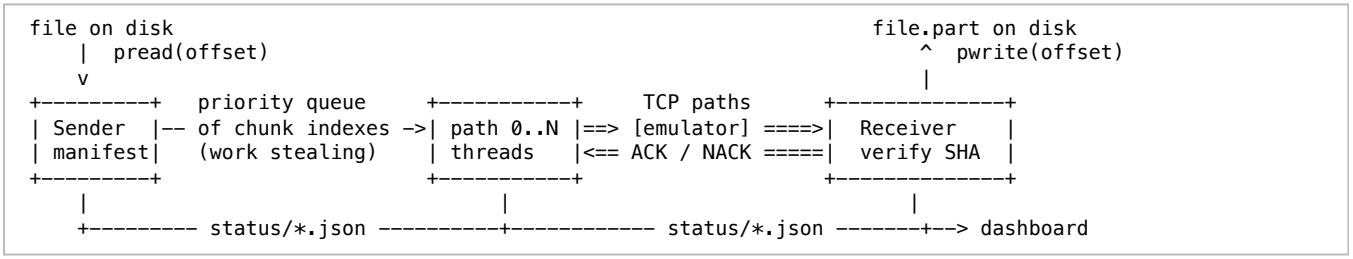
1. A frame with bad magic, version, unknown type, header CRC mismatch or `payload_len > 64 MiB` is a `ProtocolError`: the stream is out of sync, so that connection is closed. The sender treats it as a path failure.
2. A CHUNK whose index is out of range, whose length is wrong or whose SHA-256 differs from the manifest is answered with NACK; nothing is written.
3. A duplicate CHUNK (already stored) is re-ACKed and not rewritten, so a lost ACK is harmless.
4. Sender: NACK, timeout (default 5 s), reset or invalid reply requeues the chunk. Retries use exponential backoff `min(0.05 * 2^(n-1), 1.0)` s and a retried chunk goes to the front of the queue. More than 5 retries for one chunk fails the transfer.
5. A path that fails reconnects with backoff `min(0.1 * 2^fails, 2)` s; after 6 consecutive failures it is DOWN. Each connect attempt tries 4 times (0.2, 0.4, 0.8, 1.6 s waits).
6. CHUNK before MANIFEST for that session id is answered with ERROR.
7. The receiver renames `name.part` to `name` only after the whole-file SHA-256 equals the manifest value.
8. Resume: on MANIFEST the receiver re-hashes every chunk already in `name.part` and reports only those that match, so damaged or partial chunks are simply sent again.
9. The file name in the manifest is reduced to its base name before use (no path traversal).
10. Integrity layers: TCP checksum (network), header CRC-32 (framing), per-chunk SHA-256 (data, enables selective retransmit), whole-file SHA-256 (final proof).

Architecture

Components

Component	File	Role
Protocol	`mpft/protocol.py`	Header pack/unpack, CRC, `send_msg`/`recv_msg`, manifest builder
Sender	`mpft/sender.py`	One thread per path, shared chunk queue, retries, recovery metrics
Receiver	`mpft/receiver.py`	Accepts connections, verifies chunks, writes at offsets, final verify and rename
Emulator	`mpft/emulator.py`	TCP proxy per path: delay, jitter, rate limit, kill, corruption
Status	`mpft/status.py`	Writes `status/<role>-<session>.json` once a second
Metrics	`mpft/metrics.py`	Builds the result dict of a transfer
CLI	`mpft/cli.py`	`send`, `receive`, `emulate`, `dashboard`
Dashboard	`web/`	FastAPI app that reads the status files
Experiments	`experiments/`	Automated scenarios and plots

Data flow



The sender hashes the file once in a streaming pass (per-chunk hashes and the whole-file hash go into the manifest), then reads each chunk with `os.pread` only when a path takes it. The receiver writes each verified chunk with `os.pwrite` at `index * chunk_size`. Neither side holds the whole file in memory: at most one chunk per path is in RAM.

Scheduler (work stealing)

```
queue = PriorityQueue()              # items (priority, chunk_index)
for i in chunks the receiver does not hold:  queue.put((1, i))

path_worker(p):                       # runs in its own thread
    while not finished:
        if no connection: backoff, connect, (first path only) exchange MANIFEST
        (_, i) = queue.get(timeout=0.1)
        send CHUNK i; wait for reply (timeout 5 s)
        ACK            -> record bytes and RTT
        NACK           -> requeue(i)
        timeout/reset  -> requeue(i), close socket, reconnect later

requeue(i):
    attempts[i] += 1
    if attempts[i] > 5: fail the transfer
    after min(0.05 * 2^(attempts-1), 1.0) s: queue.put((0, i))    # priority 0 = before fresh chunks
```

A fast path comes back to the queue sooner, so it takes more chunks. No bandwidth estimation is needed. Measured: with paths of 80/20/5 Mbit/s the byte shares were 74 % / 20 % / 6 % (TEST_REPORT E2).

Trade-offs

Chunk size. Small chunks balance load better and waste less when a chunk is lost, but cost more headers, hashes and round trips (each path waits for an ACK per chunk). Large chunks use the link better but make the load balance coarse at the end of a transfer and make a retransmit expensive. Measured with 4 paths: 64 KiB gave 50 Mbit/s, 256 KiB 90, 1 MiB 107, 4 MiB 107 (E6). Default 1 MiB; experiments use 256 KiB.

Work stealing vs round robin. Round robin gives every path the same number of chunks, so the slowest path finishes last and the fast paths sit idle. Work stealing lets idle paths take the remaining chunks, and a dead path simply stops taking work. The cost is a shared queue with a lock, which is negligible here.

TCP per path vs UDP. TCP gives ordered, reliable bytes, congestion control and works through NAT and firewalls, so MPFT only needs framing and chunk-level recovery. A UDP design would need its own loss detection, retransmission and congestion control, and does not suit the "survive a dead path" demo as simply. The price: TCP head-of-line blocking inside one path, and one chunk in flight per path (stop-and-wait), which is why one 40 Mbit/s emulated path reaches only about 23 Mbit/s (E1). More paths hide this latency.

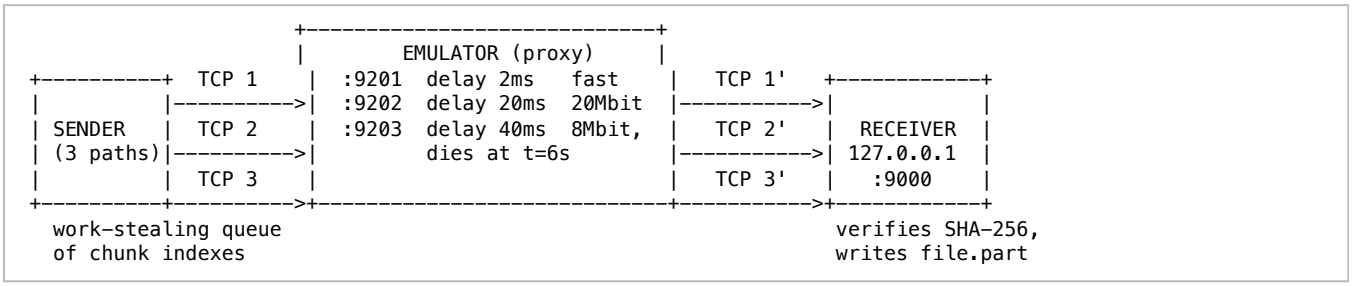
SHA-256 plus CRC-32. CRC-32 on the 16 header bytes is cheap and catches a damaged header so the receiver never trusts a wrong length. SHA-256 per chunk catches data corruption, identifies exactly which chunk to resend and is what resume uses to trust bytes already on disk. SHA-256 of the whole file proves the final result. CRC-32 is not cryptographic, so it is used only for framing.

Single-process, thread-per-path. Simple to explain and debug. Python's GIL limits CPU-bound speed, but the workload is mostly waiting on sockets and the 8-path experiment reached 172 Mbit/s.

ACK per chunk, not per byte. Application-level ACKs make "stored and verified" explicit, which TCP's ACK cannot say.

Topology and Network Emulation

Real multipath networks are not available on one laptop, so each "path" is a TCP proxy (`mpft/emulator.py`) that sits between the sender and the receiver and degrades the traffic.



Each sender path is an independent TCP connection (own 3-way handshake, own congestion window). The proxy opens a second TCP connection to the receiver per accepted connection.

Impairments (per path, JSON config)

Field	Meaning
`listen_port`	Port the sender connects to for this path
`to`	`host:port` of the real receiver
`delay_ms`	One-way delay added in each direction (RTT grows by 2x this)
`jitter_ms`	Delay varies uniformly by +/- this; byte order is never changed
`rate_mbit`	Rate limit per direction, 0 = unlimited
`drop_after_s`	Path dies this many seconds after its first connection: listener and sockets are closed, new connections are refused. 0 = never
`corrupt_rate`	Probability per forwarded 16 KiB block (sender to receiver) that one bit is flipped

Run it

```
python -m mpft receive --listen 127.0.0.1:9000 --out downloads
python -m mpft emulate --config topology.example.json
python -m mpft send bigfile.bin --to 127.0.0.1:9201,127.0.0.1:9202,127.0.0.1:9203
```

What is and is not emulated

- Emulated: latency, jitter, bandwidth limits, path failure, payload bit corruption.
- Not emulated: true IP packet loss. TCP hides loss by retransmitting, so on a real network loss shows up as lower throughput and higher RTT, which delay/rate limits approximate. Bit corruption is injected above TCP (TCP's checksum would normally catch it), to exercise MPFT's own SHA-256 and NACK path.

Test Report

Environment: macOS, Python 3.15 venv, everything on one machine (loopback). Paths are emulated by `mpft/emulator.py`. Unit tests: `pytest -q` (20 tests, about 12 s). Experiments: `python experiments/run_experiments.py` with a 50 MB random file, 256 KiB chunks unless stated; raw data in `results/*.csv`. Each experiment ran once, so small differences between rows are within noise. Default emulated path: 40 Mbit/s, 10 ms one-way delay.

1. Automated tests

ID	Scenario	Expected	Actual	Result
U1	Header is 20 bytes and round-trips	pack/unpack identical	as expected	pass
U2	Header with flipped byte	CRC error raised	`ProtocolError`	pass
U3	Header with wrong magic	rejected	`ProtocolError`	pass
U4	Message with payload over a socket	same type, session, index, payload	as expected	pass
U5	Frame delivered 1 byte at a time	reassembled correctly	as expected	pass
U6	20 MB, 4 paths, loopback	SHA-256 of output equals input, no `.part` left	match, all bytes counted	pass
U7	Chunk with one flipped bit	NACK, nothing stored; clean resend gets ACK	as expected	pass
U8	Duplicate and out-of-order chunks	all ACKed, file correct after DONE	as expected	pass
U9	Path killed after 1 MiB (proxy dies)	transfer completes via other path, failure recorded	success, hash match	pass
U10	Path goes silent (swallows data), 0.4 s timeout	timeout, other path finishes	success, hash match	pass
U11	`corrupt_rate=0.1` on 2 paths	retransmissions > 0, file correct	as expected	pass
U12	`corrupt_rate=1.0`	gives up after 5 retries, `success=False`	error mentions retries	pass
U13	Resume after interruption (half the chunks stored, receiver restarted)	only the missing half is sent	`resumed_chunks` = half, bytes = half	pass
U14	Resume when a stored chunk is damaged	damaged chunk not trusted, resent	15 of 16 resumed, hash match	pass
U15	Fast path (1 ms) vs slow path (20 ms, 8 Mbit/s)	fast path carries more than 2x slow	as expected	pass
U16	1 MiB through 8 Mbit/s limiter	takes about 1 s or more	over 0.8 s	pass
U17	Emulator `drop_after_s`	path dies, survivor finishes	success, path marked dead	pass
U18	Emulator `corrupt_rate=0.02` on both paths	transfer survives	success	pass

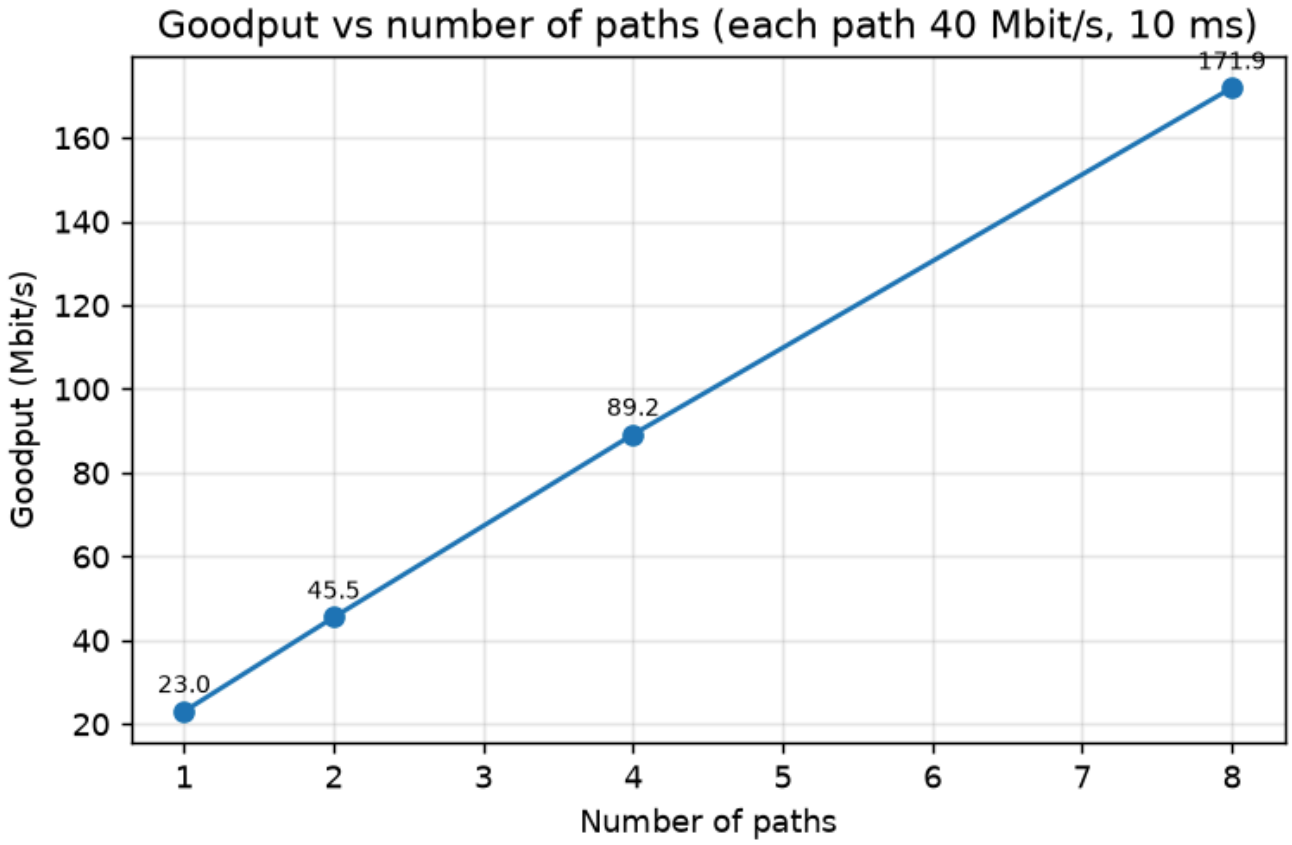
ID	Scenario	Expected	Actual	Result
U19	Dashboard API with real sender and receiver	both sessions listed, 4/4 chunks, verified	as expected	pass
U20	Dashboard with junk and old status files	junk skipped, old marked stale	as expected	pass

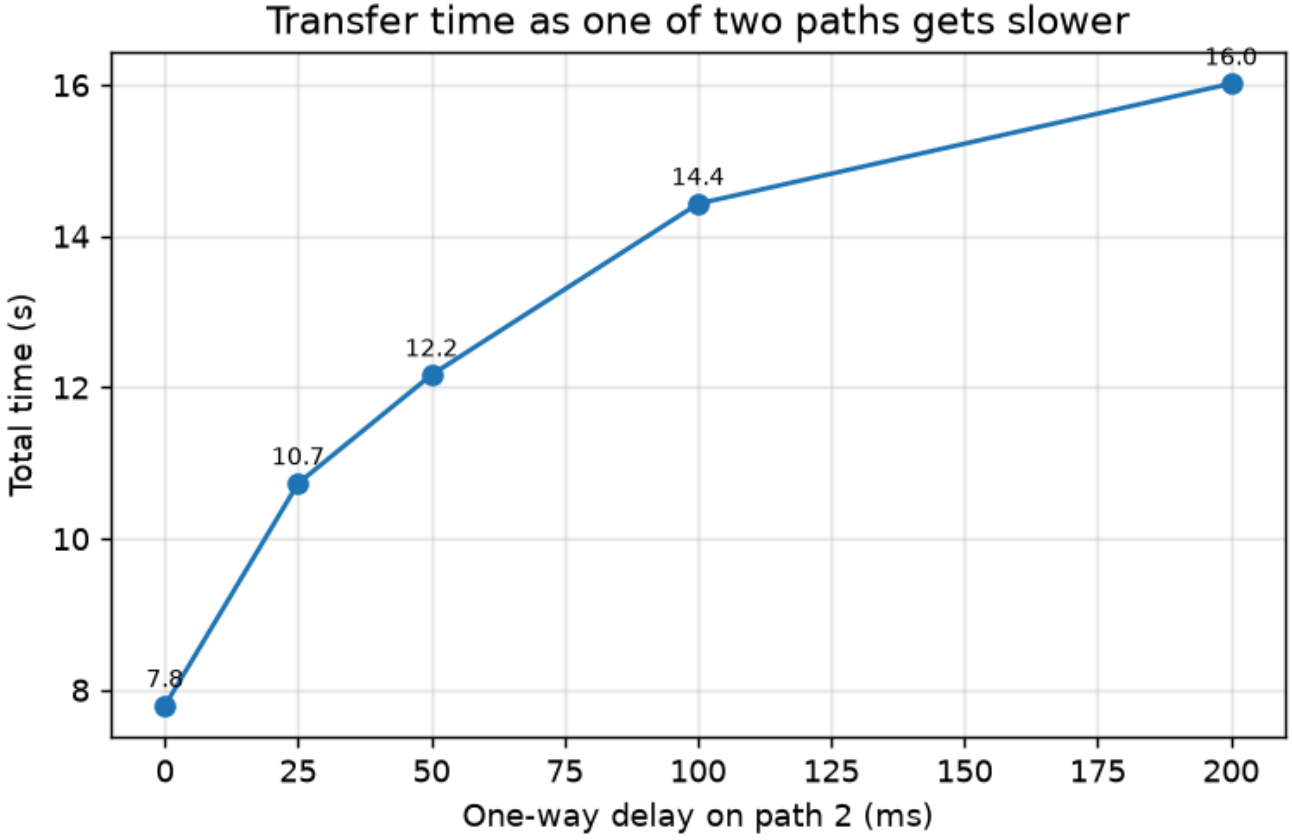
2. Experiments

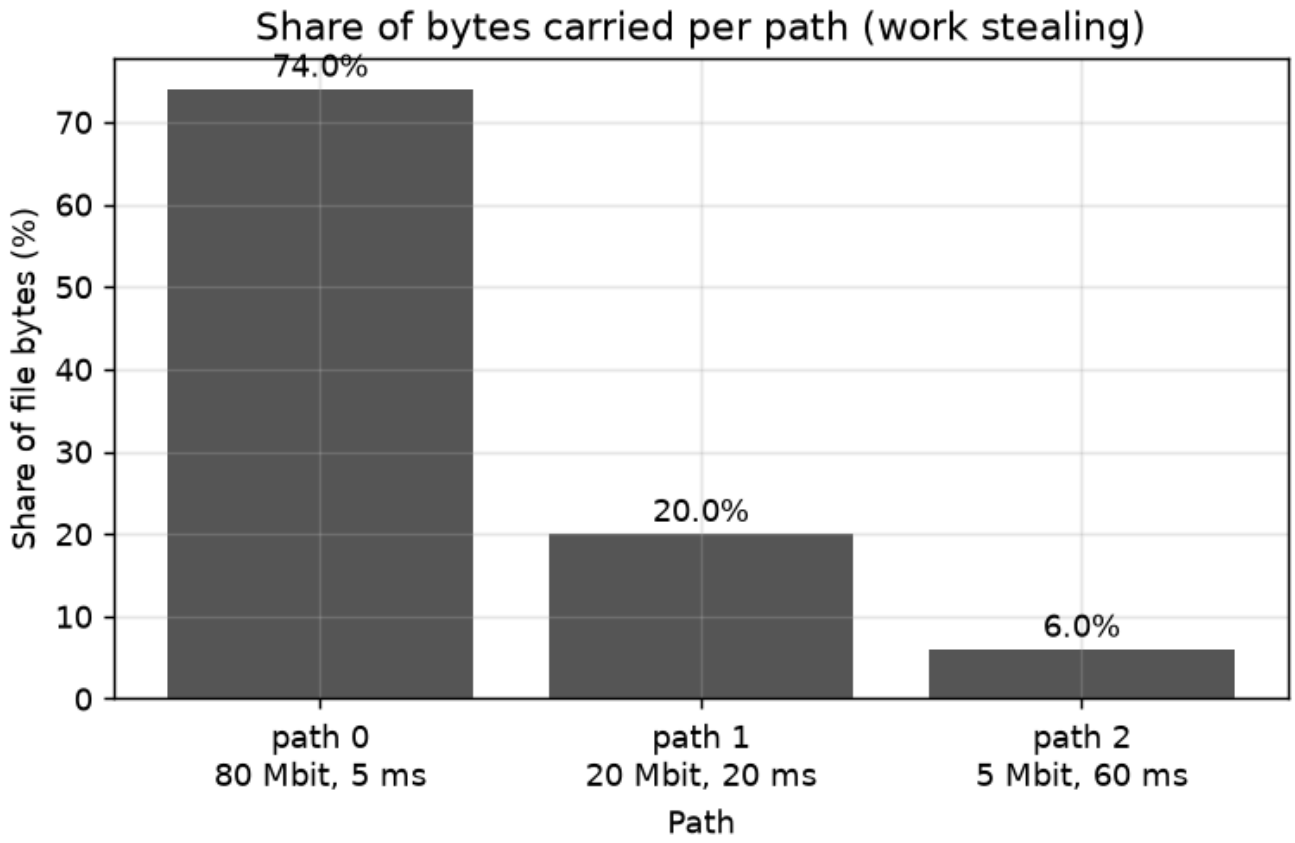
ID	Scenario	Expected	Actual (from CSV)	Result
E1	1/2/4/8 paths, equal paths	goodput grows with paths	23.0 / 45.5 / 89.2 / 171.9 Mbit/s (18.2 s down to 2.4 s)	pass
E2	Paths 80/20/5 Mbit/s	faster paths carry more	74 % / 20 % / 6 % of bytes, 7.6 s	pass
E3	2 paths, one gets 0/25/50/100/200 ms delay (+25 % jitter)	time rises, slow path share falls	7.8 / 10.7 / 12.2 / 14.4 / 16.0 s; slow path share 58 % down to 16 %	pass
E4	3 paths, path 2 dies at 1.5 s	completes, SHA-256 verified, fast recovery	success, 8.5 s vs 6.2 s without failure, 1 retransmission, recovery 0.15 s	pass
E5	Chunk corruption 0 / 1 / 5 %, 4 paths	all succeed, retransmissions near rate	0 / 2 / 12 retransmitted of 200 chunks; time 4.7 / 4.8 / 5.0 s	pass
E6	Chunk size 64 KiB / 256 KiB / 1 MiB / 4 MiB	larger chunks give higher goodput up to a point	50.2 / 90.0 / 107.0 / 107.1 Mbit/s	pass
E7	Kill sender at 40 %, run again	resumed run sends only missing chunks	killed at 3.8 s with 80/200 held; resumed run 5.7 s vs 9.2 s full, 80 chunks skipped	pass

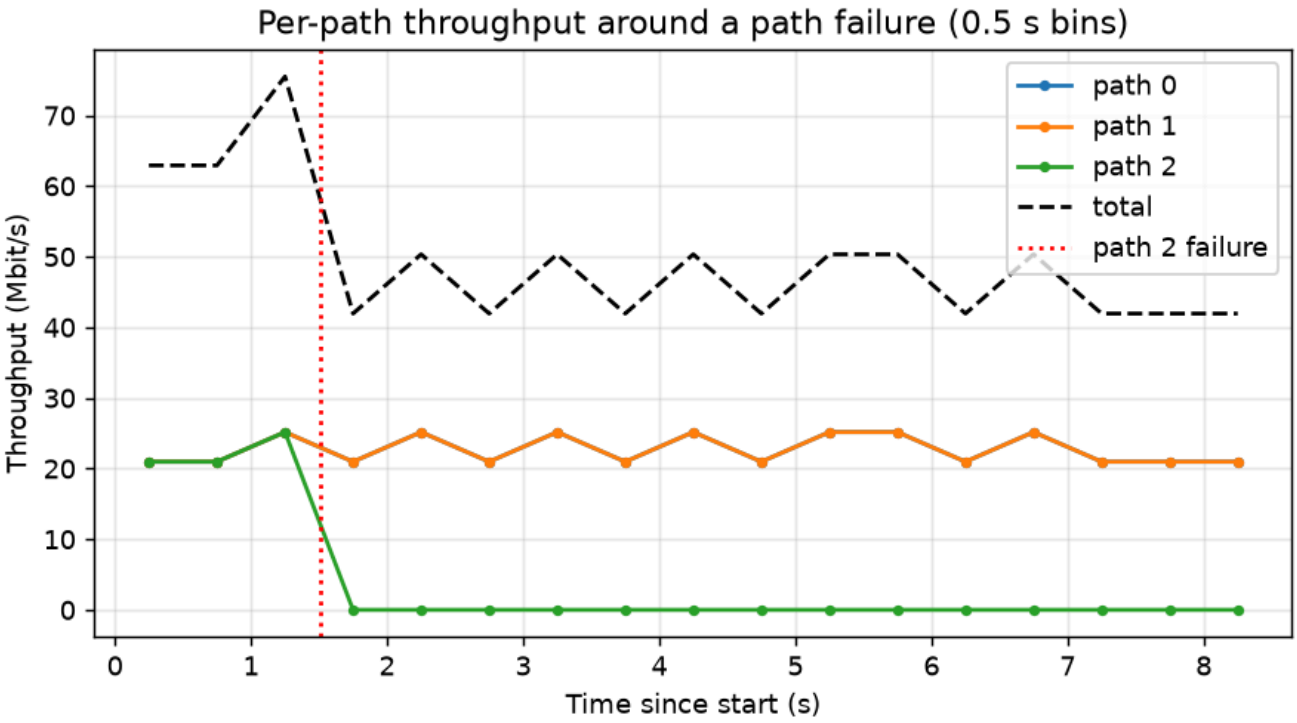
Recovery time is measured from the moment the sender detects the dead path to the moment the chunk that was in flight on it is acknowledged through another path. The emulator closes sockets, so detection is immediate; a silent black hole would add up to the chunk timeout (5 s default).

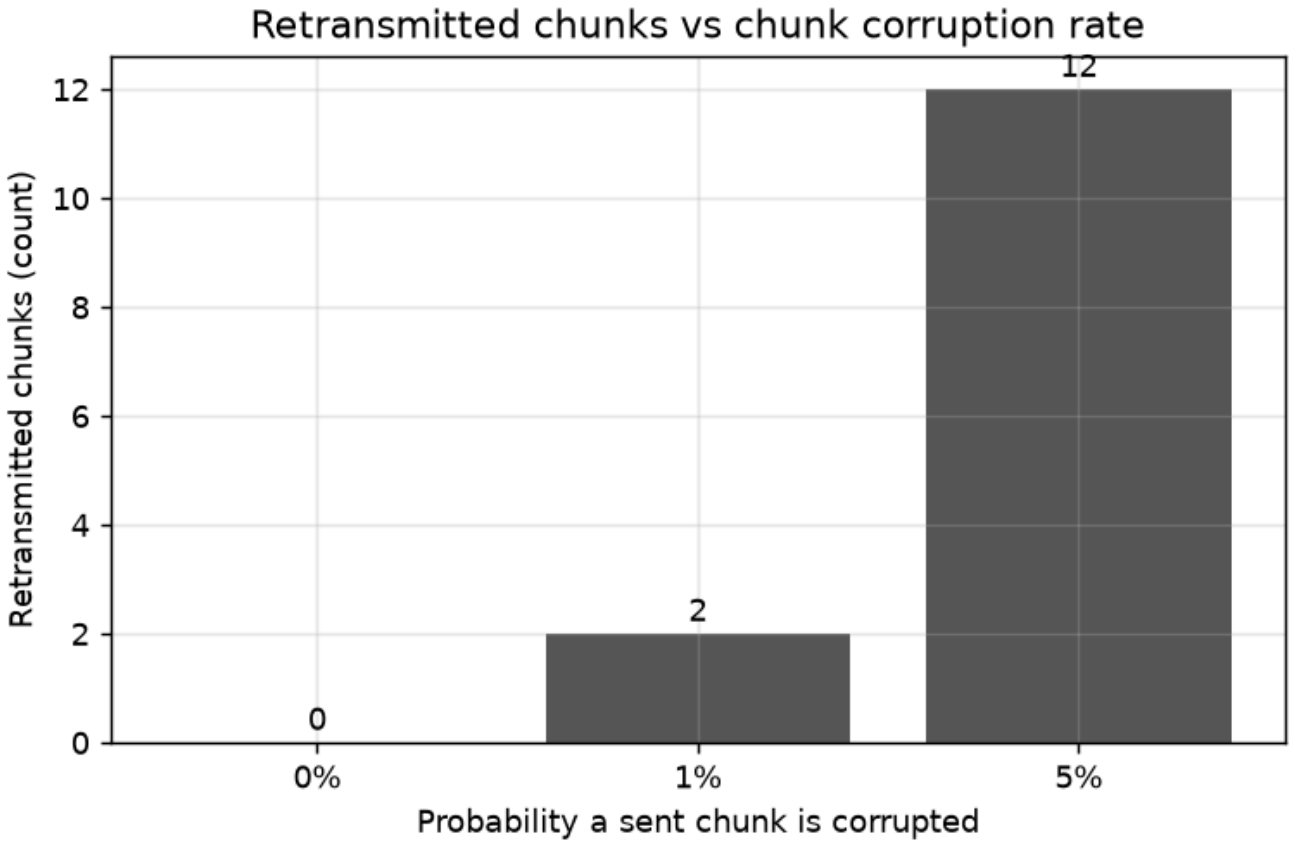
3. Graphs

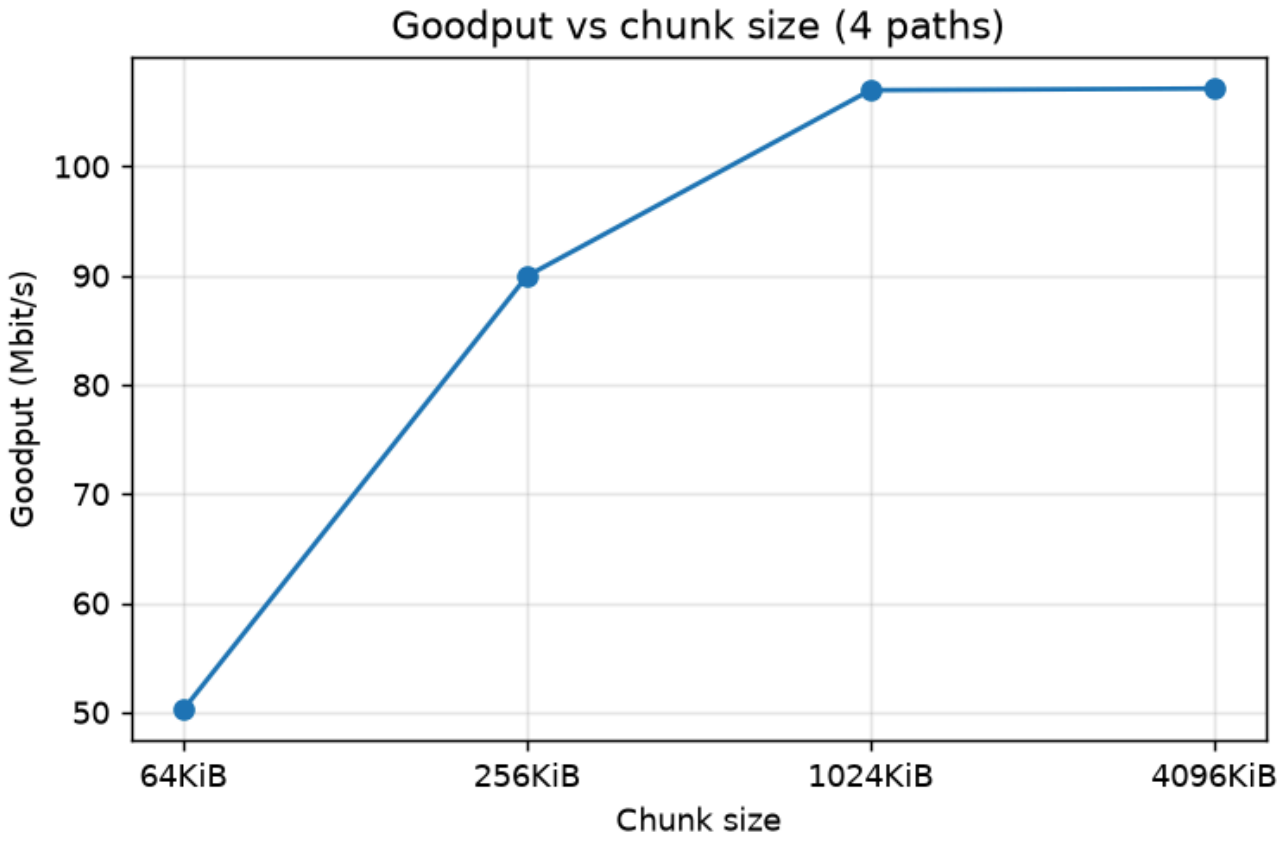

Goodput vs number of paths (each path 40 Mbit/s, 10 ms)
Goodput (Mbit/s)
Number of paths
23.0
45.5
89.2
171.9


Transfer time as one of two paths gets slower
Total time (s)
One-way delay on path 2 (ms)
7.8
10.7
12.2
14.4
16.0


Share of bytes carried per path (work stealing)
74.0%
20.0%
6.0%
Share of file bytes (%)
0
10
20
30
40
50
60
70
path 0
80 Mbit, 5 ms
path 1
20 Mbit, 20 ms
path 2
5 Mbit, 60 ms
Path


Per-path throughput around a path failure (0.5 s bins)
path 0
path 1
path 2
total
path 2 failure
Throughput (Mbit/s)
0
10
20
30
40
50
60
70
0
1
2
3
4
5
6
7
8
Time since start (s)


Retransmitted chunks vs chunk corruption rate
Retransmitted chunks (count)
0
2
4
6
8
10
12
0
2
12
0%
1%
5%
Probability a sent chunk is corrupted


Goodput vs chunk size (4 paths)
Goodput (Mbit/s)
50
60
70
80
90
100
64KiB
256KiB
1024KiB
4096KiB
Chunk size

4. Analysis

- **Scaling.** Goodput roughly doubles with each doubling of paths (E1), because each path is limited by its own rate and round trips, not by the CPU. One 40 Mbit/s path reaches only 23 Mbit/s: the protocol keeps one chunk in flight per path and waits for the ACK, so every chunk costs transmit time plus a round trip. A sliding window per path would fix this but was left out to keep the protocol explainable.
- **Work stealing works** without measuring anything (E2, E3): the slow path simply returns to the queue less often. Total time still falls short of the sum of link rates because of the per-chunk round trip above and the tail at the end, when the last chunk sits on a slow path.
- **Failure handling** (E4): losing one of three paths cost about 2.3 s of extra transfer time, which is the lost capacity, not recovery delay. The in-flight chunk was resent immediately (0.15 s).
- **Corruption** (E5): corrupted chunks were caught by SHA-256 on every run, and the cost was the retransmitted chunks only. Corruption here is injected by the sender per chunk, so the observed 1 % and 5 % are matched by 2 and 12 retransmissions out of 200.
- **Chunk size** (E6): 64 KiB loses half the throughput to per-chunk round trips; beyond 1 MiB there is no gain on this setup, while RTT per chunk and retransmit cost keep growing (mean chunk RTT 41 ms at 64 KiB, 1040 ms at 4 MiB).
- **Resume** (E7): the resumed run skipped the 80 chunks that were verified on disk.

5. Limits of this evidence

- Single machine, emulated paths, one run per setting; no confidence intervals.
- The emulator does not drop IP packets, so TCP-level loss behaviour is not tested.
- Wireshark captures and a real two-Mac run are not part of these results (see `wireshark/CAPTURE.md` and `docs/REMOTE.md`); they must be done by hand.

Demo Script (5 minutes)

Prepare before the demo (not counted): venv active (`source .venv/bin/activate`), a test file, four terminals, Wireshark open on **Loopback: lo0** with `mpft.lua` installed, a pre-captured screenshot set as backup.

```
mkdir -p /tmp/demo && head -c 60000000 /dev/urandom > /tmp/demo/video.bin   # 60 MB
cat > /tmp/demo/topo.json <<'J'
[{"listen_port":9201,"to":"127.0.0.1:9000","delay_ms":5, "rate_mbit":40},
 {"listen_port":9202,"to":"127.0.0.1:9000","delay_ms":20,"rate_mbit":20},
 {"listen_port":9203,"to":"127.0.0.1:9000","delay_ms":20,"rate_mbit":20,"drop_after_s":6}]
J
```

0:00 - 1:00 Problem and topology

Say: one TCP connection uses one route, stalls when it breaks, and a corrupted or half-sent file is wasted. MPFT splits the file into chunks, sends them over several TCP paths, verifies SHA-256, and resumes. Show `docs/TOPOLOGY.md` ASCII diagram: sender, 3 emulated paths (fast, medium, one that dies at 6 s), receiver.

1:00 - 4:00 Live transfer with a path killed

Terminal 1 (receiver):

```
python -m mpft receive --listen 127.0.0.1:9000 --out /tmp/demo/out
```

Terminal 2 (emulator, 3 paths, third dies 6 s after its first connection):

```
python -m mpft emulate --config /tmp/demo/topo.json
```

Terminal 3 (dashboard; open http://127.0.0.1:8080):

```
python -m mpft dashboard
```

Terminal 4 (sender, start Wireshark capture first):

```
python -m mpft -v send /tmp/demo/video.bin --to 127.0.0.1:9201,127.0.0.1:9202,127.0.0.1:9203 --chunk-size 256k
```

Point at: three bars on the dashboard, faster path carries more, at about 6 s the third path turns `down`, bars of the others stay up, log line "path 2 failed ... requeueing", transfer still ends with `OK`. Then prove integrity:

```
shasum -a 256 /tmp/demo/video.bin /tmp/demo/out/video.bin
```

Optional corruption run (shows NACK in the receiver log):

```
python -m mpft send /tmp/demo/video.bin --to 127.0.0.1:9201 --paths 2 --corrupt-rate 0.05
```

Note: the emulator's third path is already dead after the first run; restart the emulator (Ctrl-C, run again) before repeating.

4:00 - 5:00 Wireshark, then results

Wireshark (about 30 s): filter `mpft.type == 2` (decoded MANIFEST JSON), `mpft.type == 4` (CHUNK header fields), `tcp.flags.syn == 1` (one handshake per path), Statistics > Conversations (bytes per path). Results (about 30 s): open `results/throughput_vs_paths.png` (goodput doubles with paths) and `results/recovery_timeline.png` (one path stops, others carry on, recovery 0.15 s).

Web UI demo (about 4 minutes, one Mac)

1. Start rendezvous, alice (8000/8001) and bob (8010/8011) as in WEB.md; open bob's Receive page in tab 1, alice's Send page in tab 2.
2. Alice: choose a 100 MB file, paste bob's ID, Advanced > Network conditions = Asymmetric, Send. Bob: press Accept (point out that a code would skip this click).
3. Alice's Transfer page: chunk map filling, four paths at different Mbit/s, chart with one line per path.
4. While it runs, press Kill path 1 under Network conditions. Show the event log (path down, chunks requeued) and that the other paths finish the file.
5. Set corruption to 5 %: show "chunk N hash mismatch, retransmitting" and the Chunks failed counter.
6. Show Verification: expected and computed SHA-256 equal, "Hash verified". On bob's Receive page, Download the file and run `shasum -a 256`.
7. Copy the tcpdump command from the Wireshark panel for the next run, then filter `mpft.type == 4`.

Two-device web demo (about 5 minutes, two Macs over Tailscale)

1. Both Macs run `python -m mpft web --name NAME --ui-port 8000 --peer-port 8001 --data-port 9000 --out ./downloads`. Receiver shows Receive > Copy ID and sends it to the sender.
2. Sender: Send, pick a 200 MB file, paste the ID, Test connection (reachable), Advanced > Network conditions = Lossy path, Send. Receiver presses Accept.
3. Both screens show the same Transfer page: chunk map, paths table, graphs. Point at the receiver's screen showing the sender's events.
4. Sender kills path 1 in the Network panel. Show the incident marker "path 1 down" on the graphs, the Incidents row ("returned to the queue; path 0 delivered them") and the recovery time; press Restore and watch the path come back.
5. Set corruption to 5 %: Verification shows rejected chunk indexes. Finish: "Hash verified"; receiver Downloads and runs `shasum -a 256`.
6. History > Replay the transfer at 16x. Then show the Wireshark panel command and filter `mpft.type == 4` on the capture.

Rehearsal checklist

- [ ] Time each section with a stopwatch; cut the corruption run first if over time
- [ ] Ports 9000, 9201-9203, 8080 are free (`lsof -i :9201`)
- [ ] Web demo: ports 8000, 8001, 8010, 8011, 9100 free; latest `wireshark/mpft.lua` copied into Wireshark
- [ ] Two-device demo: both Macs on Tailscale, `--data-port 9000` allowed through the macOS firewall, Settings > self-test green on both
- [ ] Backup: screenshots of dashboard and Wireshark if the live demo fails

Viva Preparation (25 questions, answers match this code)

1. What problem does MPFT solve? A single TCP connection uses one route and the transfer dies or restarts if that route breaks or the file is damaged. MPFT sends chunks over several TCP connections in parallel, verifies each chunk with SHA-256, survives a dead path and can resume.

2. Why TCP for each path and not UDP? TCP already gives reliable ordered bytes and congestion control and passes through NAT. MPFT only adds chunking, verification and recovery at chunk level. UDP would need our own loss detection and congestion control.

3. What is in the 20-byte header? Magic `MP` (2), version (1), type (1), session id (4), chunk index (4), payload length (4), CRC-32 of the first 16 bytes (4). Defined with `struct.Struct(">2sBBIII")` plus a 4-byte CRC in `protocol.py`.

4. Why a CRC on the header if TCP has a checksum? The CRC protects the framing: a wrong `payload_len` would desynchronise the stream. It also protects against corruption injected above TCP, as our emulator does. A bad header closes the connection.

5. Why SHA-256 per chunk and for the whole file? Per chunk: identifies which chunk to resend (NACK) and lets resume trust bytes already on disk. Whole file: final proof before the `.part` file is renamed.

6. How does work stealing work here? All paths pull indexes from one shared `queue.PriorityQueue`. A path takes a new index only after its previous chunk was ACKed, so a faster path takes more. Measured: with 80/20/5 Mbit/s paths the byte shares were 74/20/6 %.

7. Why work stealing and not round robin? Round robin gives each path equal work, so the slowest path finishes last and a dead path blocks its share. Work stealing adapts without measuring bandwidth.

8. What happens when a path dies mid-transfer? The worker's send or receive raises an error (reset, timeout, bad frame). `fail_path` records the failure time and `requeue` puts the in-flight chunk back with priority 0. The worker closes the socket, backs off and reconnects; after 6 consecutive failures, or if connect fails, the path stays down and the others finish.

9. What are the retry rules? A chunk may be retried 5 times (`MAX_RETRIES`); the 6th failure fails the transfer. Delay before re-queue is `min(0.05*2^(n-1), 1.0)` s. A path reconnects with `min(0.1*2^fails, 2)` s backoff. Per-chunk timeout is 5 s.

10. Why do retried chunks go to the front of the queue? With a normal FIFO queue the retried chunk waited behind all fresh chunks; recovery time was 6.9 s instead of 0.2 s in the failure experiment. The `PriorityQueue` with `(0, idx)` for retries fixed it.

11. How does resume work? On MANIFEST the receiver opens `name.part`, and if it exists with the right size it re-hashes each chunk on disk against the manifest. The MANIFEST_ACK lists the chunks that match, and the sender queues only the rest. A damaged chunk simply fails the hash and is resent.

12. Why not keep a resume state file? Extra state can disagree with the file after a crash. Re-hashing costs one read of the partial file but is always correct.

13. How is the file written without loading it into RAM? Sender: `os.pread` of one chunk when a path takes it, and the manifest is built in one streaming pass. Receiver: `os.pwrite` at `index*chunk_size`, which is thread safe and allows out-of-order chunks.

14. What does the receiver do with duplicate or out-of-order chunks? Out of order: written at its own offset. Duplicate: already in the `held` set, so it is ACKed again and not rewritten (counted in `duplicates`). This makes a lost ACK harmless.

15. When is the file considered complete? When `held` has all chunks the receiver hashes the whole `.part` file and, if it equals `file_sha256`, renames it. The sender then sends DONE after all ACKs and the receiver replies DONE (or ERROR). The rename happens only after verification.

16. Why is DONE a handshake in both directions? The sender cannot know the file was verified just from chunk ACKs. Sending DONE after all ACKs gives a deterministic point: the receiver confirms verification with DONE or reports ERROR.

17. How is the first MANIFEST sent if path 0 is down? `ensure_manifest` uses a lock: the first path that connects sends the MANIFEST and the rest wait. If it fails, the lock is released and another path does it.

18. How is recovery time measured? From the first failure detected (`t_fail`) until every chunk that was in flight on failed paths has been ACKed. In the experiment: 0.15 s. With a silent black hole it would include the chunk timeout.

19. Why does one 40 Mbit/s path only reach about 23 Mbit/s? One chunk in flight per path (stop-and-wait): each 256 KiB chunk costs transmit time plus a round trip before the next. Several paths overlap these waits; larger chunks reduce the relative cost (1 MiB gave 107 Mbit/s on 4 paths vs 90 at 256 KiB).

20. How does the emulator add delay without reordering bytes? Each direction has a reader thread that stamps blocks with a due time (`max(previous_due, now+delay+jitter)`) and a writer thread that sends at that time. Due times never decrease, so order is kept. Rate limit: the writer sleeps `len*8/rate` after each block.

21. What does the emulator NOT do? It does not drop IP packets (TCP would hide that anyway) and corruption is injected above TCP. It only approximates real networks; everything runs on one machine.

22. How do you guarantee corrupted data is never accepted? Every CHUNK is hashed on arrival and compared to the manifest before `pwrite`; a mismatch gives NACK and nothing is stored. The whole-file hash is a second barrier. Tested with `corrupt_rate` 1 %, 5 %, 10 % and by flipping a bit directly.

23. What security does MPFT have? None by default: no encryption or authentication. The manifest file name is reduced to its base name to prevent path traversal, and header length is capped at 64 MiB. Use Tailscale or a VPN for confidentiality.

24. How does the dashboard get its data? Sender and receiver write `status/<role>-<session>.json` atomically (temp file plus `os.replace`) every second. FastAPI's `/api/status` reads the files and marks those not updated for 5 s as stale; the JS page polls every second.

25. What would you improve? A sliding window per path (several chunks in flight), TLS or a keyed MAC, adaptive chunk size, true packet-loss emulation, and resume metadata that survives a sender restart without re-hashing the receiver's file.

Web layer

26. Why two servers per instance? The UI server can start transfers, read local files' uploads and kill emulator paths, so it listens on 127.0.0.1 and checks Host and Origin. The peer server must be reachable by other machines, so it is a separate app on 0.0.0.0 that exposes only `GET /health` and `POST /offer`.

27. How does a Receiver ID work without a server? It is base32 of `host:peer_port`, so the sender decodes it locally. The host is the Tailscale 100.x address if present. A code needs the rendezvous service because six random characters cannot encode an address.

28. What makes a code safe to use? It is generated with `secrets`, has no confusable letters, expires in 10 minutes, is deleted on first use, and failed attempts are limited to five per minute per client. It does not provide encryption.

29. Why does an offer need Accept? Anyone who can reach the peer port could otherwise push files to the receiver. A valid code proves the sender was given the code by the receiver, so that offer is trusted; an ID alone is not a secret.

30. How do the live panels get real data? `mpft/events.py` is a thread-safe bus that sender, receiver and emulator emit into. The UI server subscribes, stores events in a ring buffer and streams them over Server-Sent Events; the page reduces them into state (`MP.reduce`) and redraws. A reload replays the buffer.

31. How does Kill path work, and is it a real failure? The emulator proxies are real TCP listeners in front of the receiver. Killing one closes its sockets, so the sender sees a reset or timeout exactly as in the earlier experiments, requeues the chunk and the other paths finish it. The failure is real TCP, only the network is emulated.

32. How do you avoid overwriting or traversal on the receiver? Offer names are reduced to a safe base name; data arrives in a staging directory and is renamed to a unique name (`name (1).ext`) under a lock. Downloads are served only for names the node recorded, and the resolved path must be inside the output directory.

33. What does the web layer not protect against? There is no encryption or authentication on the peer API or rendezvous service; anyone on the network who knows the receiver address can send an offer (it waits for Accept) and a rogue rendezvous could map a code to the wrong ID. Tailscale limits who is on the network and encrypts it.

34. Why does the sender relay its events instead of the receiver pulling them? The sender can reach the receiver (it just made the offer) but the receiver may not reach the sender behind NAT. The

sender POSTs batches to `/events/{id}` and the reply carries the receiver's new events, so one connection direction gives both people the same log.

35. How do you merge two machines' clocks? During the offer round trip the sender takes t0 and t3, the receiver reports its arrival t1 and reply t2; offset = ((t1-t0)+(t2-t3))/2. The receiver's wait for Accept cancels out. Events keep a per-side sequence number, so order is by corrected time, then side, then sequence; clock error can blur ordering across sides by roughly the network delay but never within a side.

36. What is the session token for? It is returned only in the accept reply and required on `/events`, so a stranger who finds the peer port cannot inject events into someone's dashboard. It is compared in constant time and events are forced to the side the token holder may speak for.

37. How does one Receiver ID work over two networks? It encodes all of the receiver's addresses (Tailscale and LAN). The sender probes each, uses the reachable ones, and assigns paths round-robin, so a transfer can use both networks at once. Advanced lets you force one.

38. What happens if the receiver disappears mid-transfer? Paths fail, chunks are requeued up to the retry limit, the job fails with a message to send again. The receiver keeps the `.part` file in a staging folder named by the file hash, and the next offer of the same file resumes from the chunks whose hashes match.

39. How is the incident table built? A reducer watches the merged events: `path_down` opens an incident, `requeue` events attach chunks to it, `backoff` adds retry delays, and when every attached chunk is acknowledged on another path the incident is recovered; recovery time and the paths that absorbed the work come from those events.

40. Why are path ids not merged between sender and receiver? The receiver numbers every accepted connection, so a reconnect gets a new id, while the sender keeps its own. The dashboard therefore uses only the sender's path events for the Paths table and counts a chunk as done when the sender sees the ACK.

41. How does the chunk map scale to 20,000 chunks? It is a canvas, not DOM elements; above 6000 chunks several chunks share one cell and the cell shows the group's state. Hover and click report exact values for single chunks.

42. How does Replay work? Both sides write every tagged event to `sessions/<id>.jsonl`. Replay loads that file and feeds events to the same reducer used live, up to a virtual clock; scrubbing backwards resets and re-reduces. There is no separate replay code path to drift from the live view.

43. What would you do for real security? Encrypt and authenticate the data plane (TLS or Noise) and the peer API, bind the session token to a key exchange, and replace code lookup by a verified rendezvous. Today Tailscale supplies encryption and device authentication.

Contributions

Template only. Fill in the real names and what each person did. Do not leave placeholders in the submitted version.

Member	Name	Student ID	Contribution (fill in)
1	*fill in name*	*fill in*	*fill in*
2	*fill in name*	*fill in*	*fill in*

Suggested module split for the teammate who joins

Each person should be able to explain their modules in the viva using `docs/WALKTHROUGH.md`.

Teammate takes over	Files	Why it is a good unit
Emulator and experiments	`mpft/emulator.py`, `experiments/`, `docs/TOPOLOGY.md`, `results/`	Self-contained, easy to demonstrate, produces the graphs
Wireshark and remote run	`wireshark/`, `docs/REMOTE.md`, screenshots	Hands-on, needs the real two-Mac test
Dashboard	`web/`, `mpft/status.py`	Small and visual

Remaining with the original author: `protocol.py`, `sender.py`, `receiver.py`, `cli.py`, `docs/PROTOCOL.md`, `docs/ARCHITECTURE.md`.

Git evidence

`git shortlog -sn` shows commits per author; the teammate should commit under their own identity (`git config user.name`, `git config user.email`).